

OS Development

3. Programming model of the x86_64 processor architecture

Prof. Dr. Michael Schöttner

Dr. Fabian Ruhland

- History

- x86 programming model

- Address translation

- Protection

- Tasks

- Summary

- 8086 (1978) - Prime father of the PC processor
- 80286 (1982) - Segmentation based memory protection
- 80386 (1985) - Paging → Memory protection based on pages
- Pentium (1993) - superscalar, 64-bit data bus, MMX, APIC
- Pentium II (1997) - RISC-like micro instructions
- Pentium III (1999) - SSE
- Pentium 4 (2000) - SSE2, Hyperthreading, Intel 64 / EM64T
- Core (2005) - Dual Core
- Core 2 (2006) - Quad Core, 64 bit (x86_64)
- Has-/Broad-well (2013) - AVX2, TSX, FMA3
- Rocket / Tiger Lake (2021) - DL-Boost, Memory encryption
- Alder Lake (2021) - Division in P- and E-cores (max. 8 + 8)

Advanced Vector Extensions (based on SSE)
Transactional Synchronization Extensions
Fused-Multiply-Add Instructions

Deep-Learning (DL) Boost
→ AVX extension

P = Performance
E = Efficiency
Hybrid Cores

- History

- x86 programming model

- Address translation

- Protection

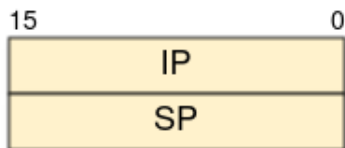
- Tasks

- Summary

- 16-bit architecture, little endian
- 20-bit address bus → 1 MiB max. physical memory
- Few registers (from today's standpoint)
- 123 instructions (1-4 byte length)
- Segmented memory (64 KiB segments)
- Still relevant today
 - On multicore systems, other cores startup in real mode

8086: Registers

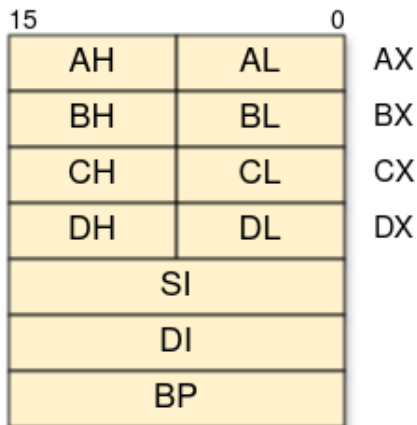
Instruction and Stack Pointer



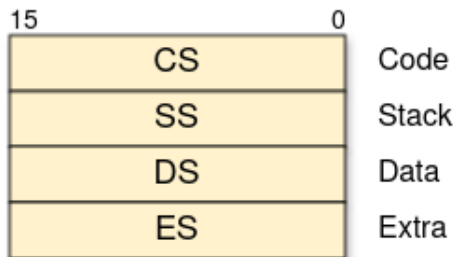
Flags Register



General Purpose Register

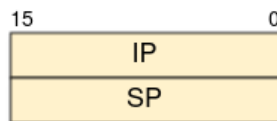


Instruction and Stack Pointer

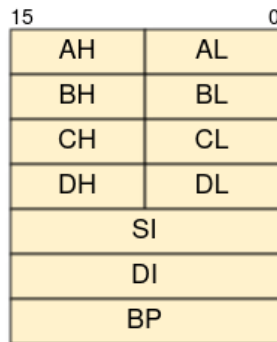


8086: Registers

Instruction and Stack Pointer



General Purpose Register



AX: Accumulator Register

- arithmetic/logic operations
- I/O
- shortest machine code

BX: Base Address Register

CX: Count Register

- `LOOP` instruction
- *String* operations using `REP`
- Bit `shift` and `rotate` operations

DX: Data Register

- `DX:AX` used as 32-bit values for `MUL/`
`DIV`
- Port numbers for `IN` and `OUT`

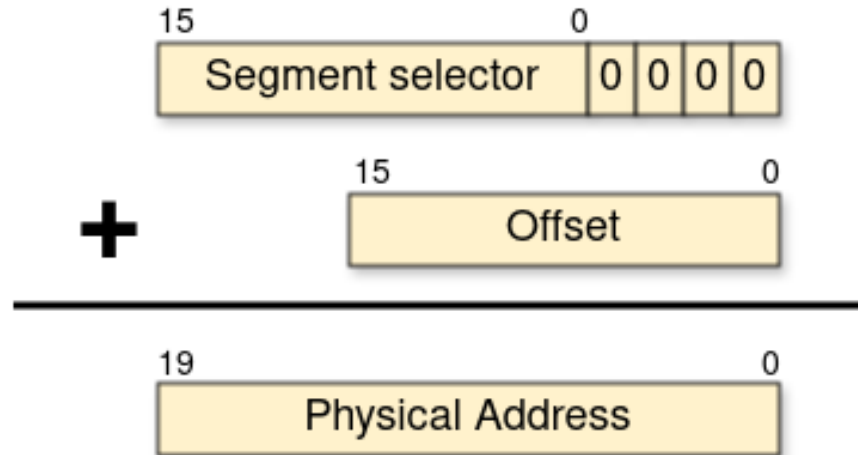
SI/DI: Index Register

- Displacement for array operations

BP: Base Pointer

8086: Segmented Memory

- Logical addresses consist of:
 - Segment selector (determined by a segment register)
 - Offset (usually given by a general purpose register or directly encoded into the instruction)
 - Physical address calculation:

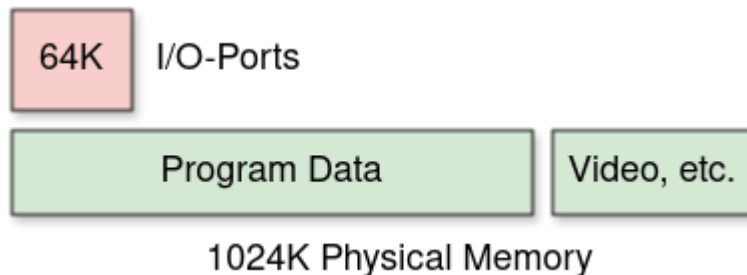


8086: Segmented Memory



8086: I/O-Port vs. Memory Addresses

- I/O-port and memory addresses:
 - Differentiation through CPU pin *M/I/O*.
 - 16 address lanes for I/O → 16-bit
 - Still supported today (even in Protected Mode and Long Mode)
- Additionally: Memory Mapped I/O (e.g., for video cards)
 - Memory access goes to a device instead of memory



- First IA-32 CPU was the Intel 80386
 - The term *IA-32* was introduced much later
- 32-bit technology: register, data und address bus
 - From Pentium Pro: 64 Bit data und 36 bit address bus
- Complex protection and multitasking support
 - Protected Mode
 - Originally introduced with the 80286 (16-Bit)
- Compatibility
 - With older operating systems through Real Mode
 - With older applications through *Virtual 8086 Mode*
- Segmentation based memory model (mostly used as *Flat Memory*)
- Page based *Memory Management Unit* (MMU)

- x86-64: Original name of 64 bit extension by AMD
 - Sometimes abbreviated as *x64*
 - Later replaced by *AMD64*
- Intel licensed x86-64 from AMD
- Intel 64 is Intel's own label (for the same architecture)
 - Minimal differences in instruction set compared to AMD64
 - Completely different than the own *IA-64* (Itanium) architecture

- Additional registers (all 64 bit long)
- Paging must be enabled
- Segmentation strongly restricted
- New feature (among others): *No Execute* protection per page

- Real Mode = 16 bit addressing
- Protected Mode = 32 bit addressing (also called IA-32)
- Long Mode = 64 Bit addressing (also called IA-32e)
 - *e* meaning *extension*
 - Used in the Intel manuals
 - To differentiate against IA-64
- (Compatibility Mode)
 - Supporting Protected Mode applications in Long Mode

x86-64: Registers

General-purpose registers

	63	16	15	0
RAX				AX
RBX				BX
RCX				CX
RDX				DX
RSI				SI
RDI				DI
RBP				BP
R8				
⋮				
R15				

Instruction and stack pointer

	63	16	15	0
RIP				IP
RSP				SP

Status register

	63	16	15	0
RFLAGS				FLAGS

Segment registers

	15	0		15	0
Code	CS		Extra	FS	
Stack	SS		Extra	GS	
Data	DS				
Extra	ES				

Memory-management registers

	15	0 63	0 31	0
TR	TSS sel.	TSS Base Address	TSS Limit	
LDTR	LDT sel.	LDT Base Address	LDT Limit	
IDTR		IDT Base Address	IDT Limit	
GDTR		GDT Base Address	GDT Limit	
			15	0

Details follow ...

Control registers

	63	32 31	0
CR8			
CR4			
CR3			
CR2			
CR0			

Debug registers

	63	32 31	0
DR0			
DR1			
DR2			
DR3			
DR6			
DR7			

Model-specific registers (MSRs)

- History
- x86 programming model
- Address translation
- Protection
- Tasks
- Summary

- Effective addresses (EA) are calculated using the following scheme
 - All general purpose registers can be used

$$\text{EA} := \text{Base-Reg.} + (\text{Index-Reg.} * \text{Scale}) + \text{Displacement}$$

1/2/4/8

1/2/4 bytes

EA

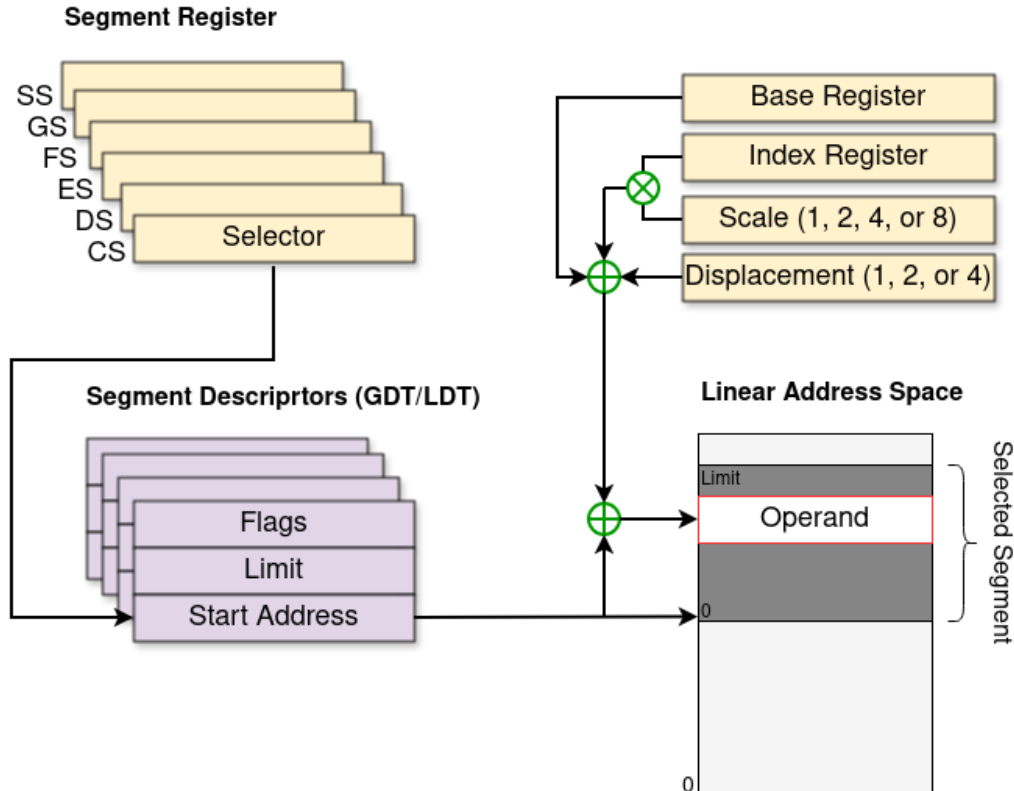
- Example: `MOV RAX, array[RSI * 4]`
 - Lesen aus einem Array mit 4 Byte großen Elementen und RSI als Index
- New in x86-64: Addressing relative to instruction pointer

$$\text{EA} := \text{RIP} + \text{Displacement}$$

- Each segment has a start address and a length
- Segment registers store *segment selectors* = Index into the GDT/LDT
 - LDT was rarely used by operating systems
- A program (in execution) consists of multiple memory segments
 - At least CODE, DATA and STACK
- *Linear address* is segment start address + effective address
 - Effective address = Value in a general purpose register
 - Linear address = physical address, if the paging unit is turned off

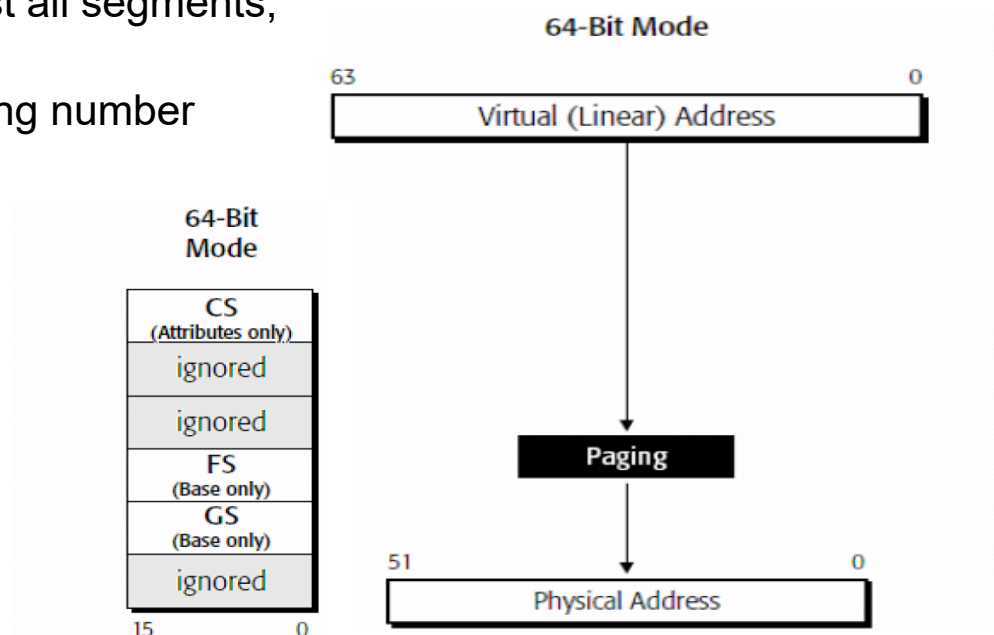
- Flat memory:
 - Practically all modern 32-bit operating systems set all segments to start address 0
 - This way, segments do not influence address calculation
 - Furthermore, only four GDT entry are used (CODE + DATA, each for *User* and *Kernel Mode*); LDT remains completely unused
- Segmentation cannot be turned off (because of protection), but Paging is not required in IA-32 Protected Mode

Protected Mode – Segments



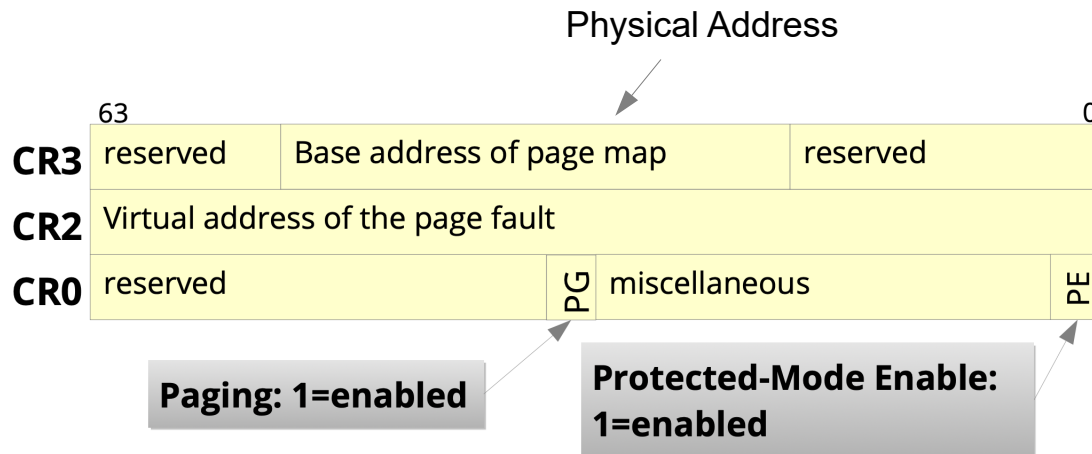
Long Mode: Segmentation

- 16-bit Selectors
- Start addresses are ignored in almost all segments, except FS and GS
- No limit check; essentially only the ring number in CS remains relevant (= CPL)



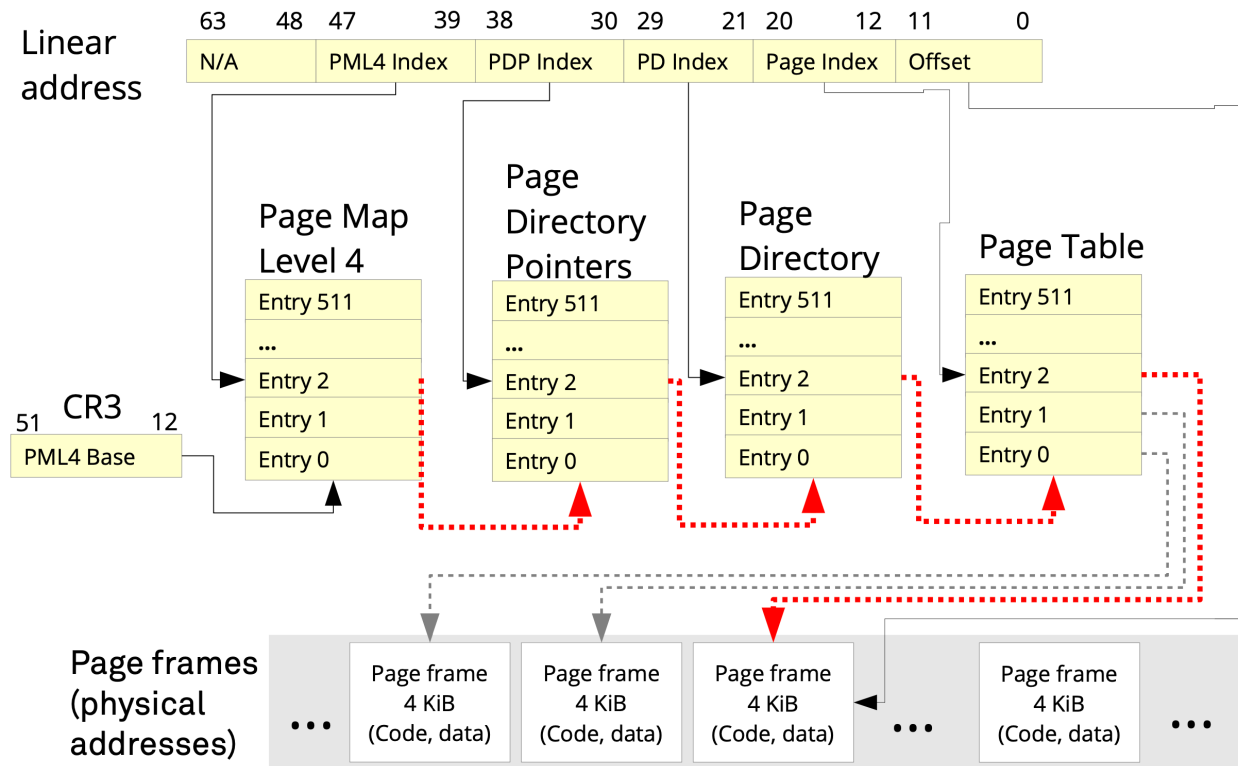
Long Mode: Paging

- Paging is mandatory in Long Mode
- The basic paging configuration is handled by the CRx **C**ontrol **R**egisters



Long Mode: Page Tables

- Four or five levels of page tables



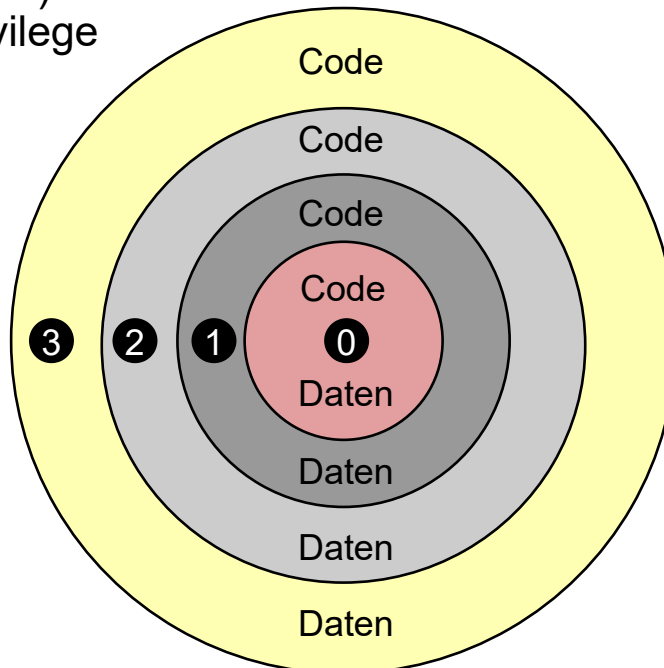
- Problem: Paging slows down memory accesses, since the page tables need to be accessed each time
- Solution: *Translation Lookaside Buffer* (TLB):
 - Fully associative cache
 - Tag: consists of page table indices
 - Data: Page Frame address
 - Size (80386): 32 entries
- With normal applications, the TLB reaches a hit rate of ~98%
- Remarks:
 - Writing the CR3 register invalidates the TLB
 - Not anymore since Intel Westmere (2010) → TLB Tag has additional 12-bit *Process Context ID* (PCID)
 - When protection/access bits in a page table entry are modified, the corresponding TLB entry must be deleted (INVLPG instruction)

- History
- x86 programming model
- Address translation
- Protection
- Tasks
- Summary

- Goals:
 - Protection against unauthorized memory accesses and code execution
 - Protection against kernel crashes or crashes of other processes

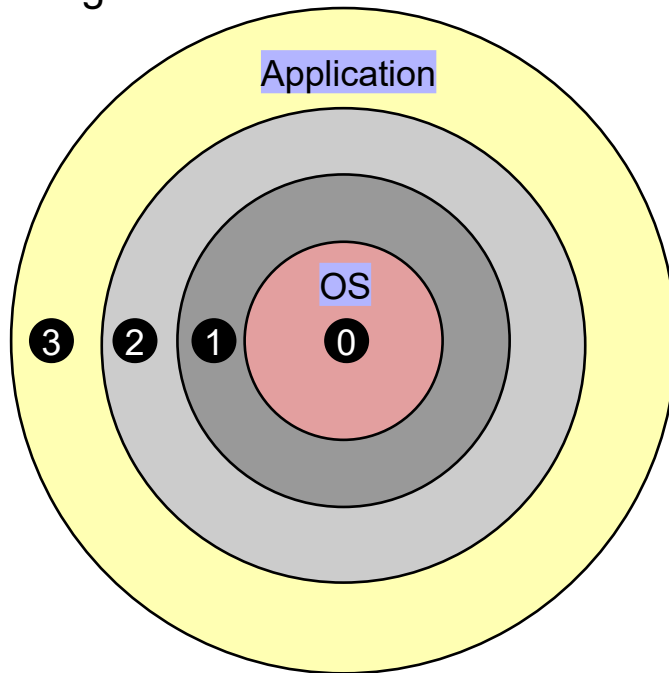
- Requirements: Code and Data...
 - are classified in regard to their trustability
 - and this classification is enforced by the operating system, with the help of the process

- Each segment descriptor (Code/Data) has two bits, determining which privilege level the segment is assigned to
 - Up to four privilege levels possible
 - In practice, only two are used



Protection Rings

- Smaller ring numbers grant more privileges
- Ring 3 is meant for applications
 - = User Mode
- Ring 0 is meant for the OS
 - = Kernel/Supervisor Mode



Protection Rings and Gates

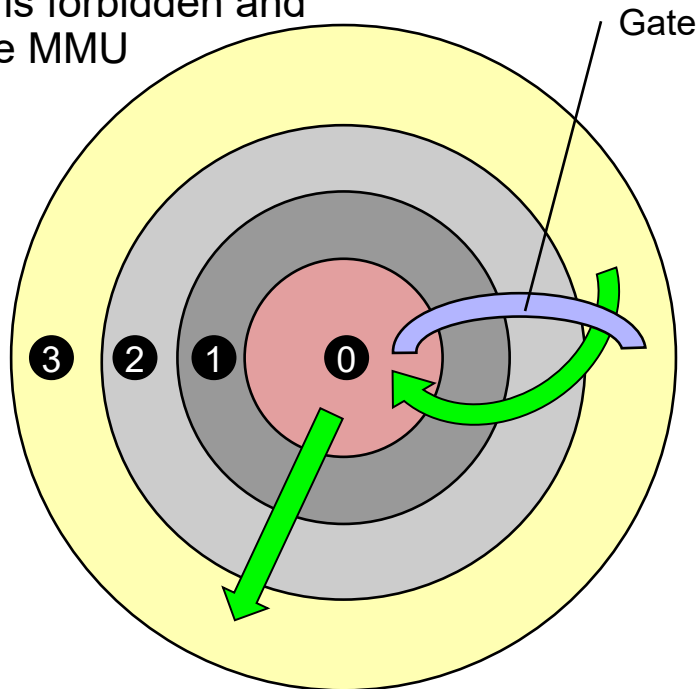
- Accessing memory of an inner ring is forbidden and such accesses are prohibited by the MMU

- General Protection Fault

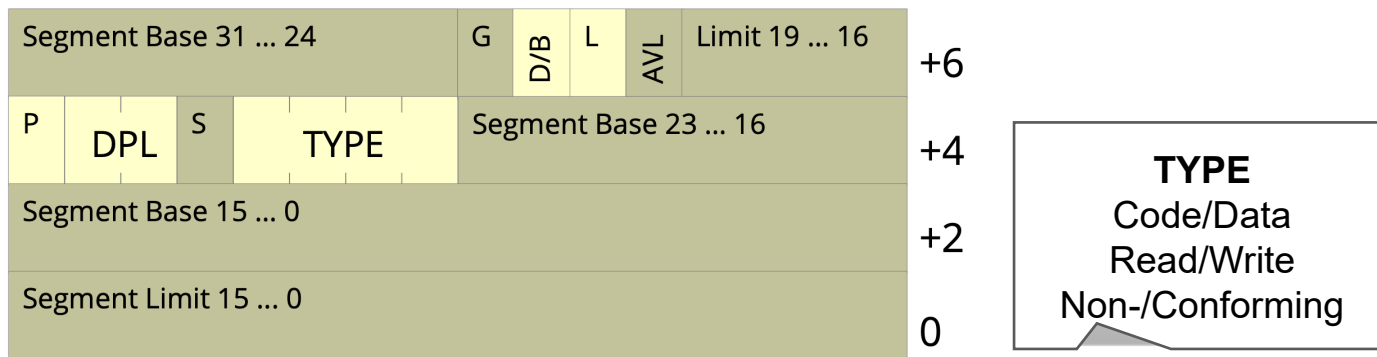
- Code of an inner ring can only be called via **Gates**

- Allow direct calls from ring 3 to ring 0

- Memory accesses and calls to outer rings are always allowed



- Are entries in the Global Descriptor Table (GDT) and allow code protection
 - (IA-32: additional protection via limit (= end of segment))
 - 8 byte per entry (Exception: system segments use 16 byte in x86_64)



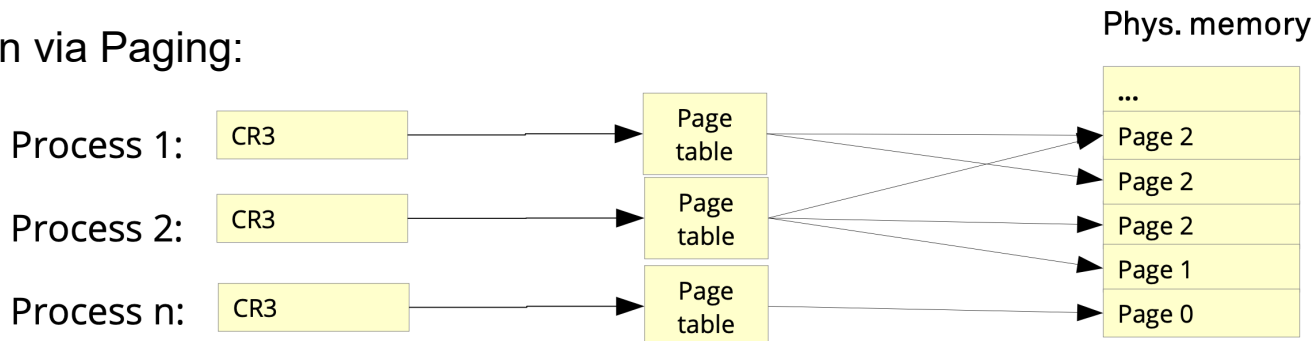
System Segments
(Call-Gate, Task
State Segment)

P - Present Bit
DPL - Descriptor Privilege Level
S - System Segment

G - Granularity
D/B - 16/32 Bit Seg.
L - Long Mode aktiv

- Most 32-bit PC operating systems have not used segmentation
 - 32-bit effective address = linear address
- In Long Mode (64 bit) the segment base address is ignored by the processor

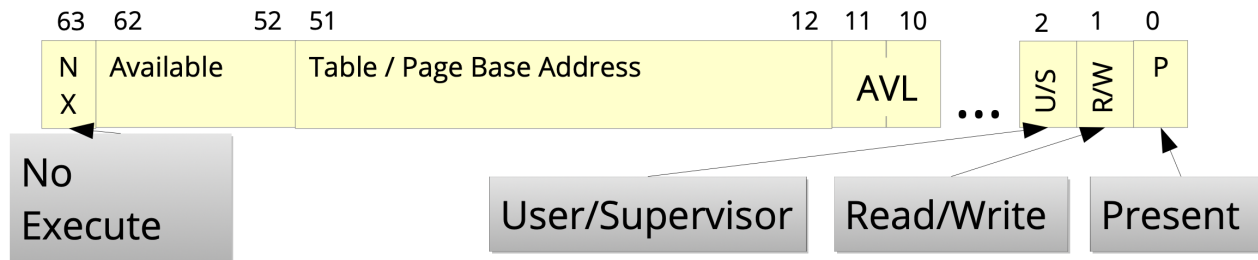
- Protection via Paging:



- Each process only sees its own *virtual address space*
 - Page level access control (see next slides)

Page Table Entry

- Flags are settable on all levels of the page table hierarchy
- Protection bits:
 - R/W = Read/Write → Protection against write accesses
 - NX = No eXecute → Do not execute code inside data
 - U/S = User/Supervisor → Access allowed for all or only in ring 0



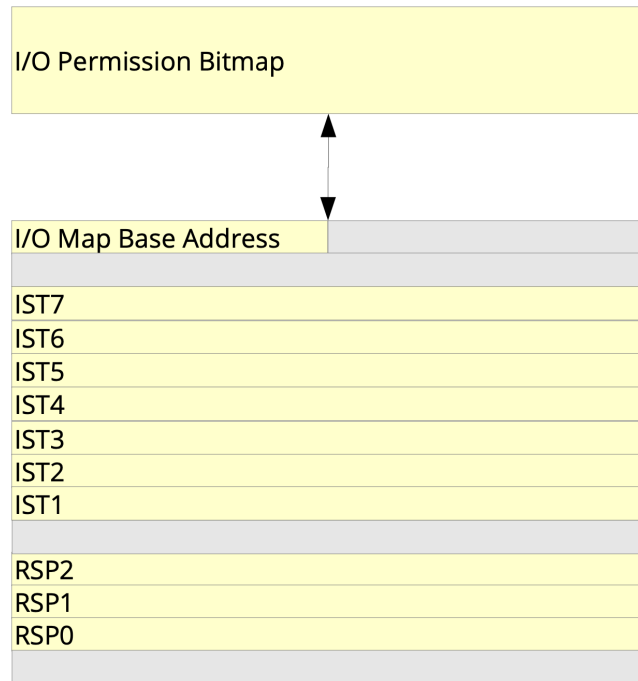
- Available = AVL = These bits are free to use by the operating system

- Modern 64-bit operating systems only need the following GDT entries
 - One data segment descriptor
 - Two code segment descriptors (one with DPL = 0 and one with DPL = 3)
- Protection is enforced via Paging
 - Protection bits (NX, U/S, R/W)
 - Page Tables:
 - Which pages are present/accessible?

- History
- x86 programming model
- Address translation
- Protection
- **Tasks**
- Summary

- State of a task is stored in a *Task State Segment* (TSS)
 - Stores all registers
 - Stack pointer for each ring
 - CR3 register
 - I/O permission bitmap (see next slides)
- Current TSS determined by *Task Register* (TR) of a CPU core
 - Each core has its own TR
- Task switch with hardware support via a *Task Gate*
 - Not used in most operating systems and instead implemented in software

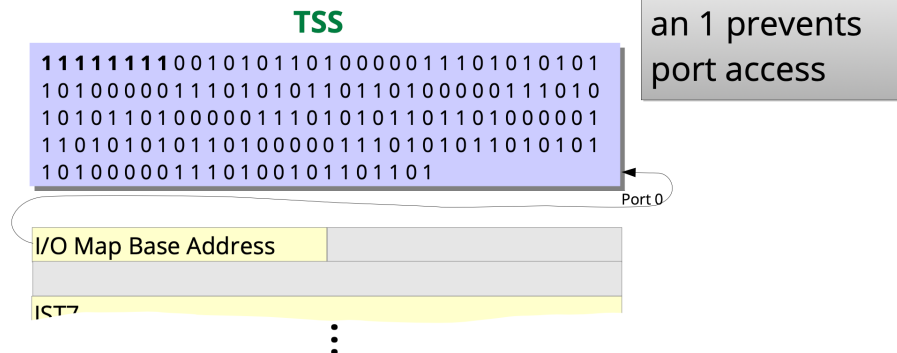
- No HW support for task switching
- TSS existent, but only stores:
 - Stack pointer per ring
 - IST = *Interrupt Stack Table*
 - I/O permission bitmap (see next slide)
- IST allows separate stacks for interrupt handlers
 - IST number can be specified in entries inside the Interrupt-Descriptor Table (IDT)



Long Mode: I/O Permission Bitmap

- Access protection for I/O-ports (only little relevance nowadays)
 - I/O privilege level (IOPL) represented by two bits inside the RFLAGS register
 - Port access is granted if $CPL \leq IOPL$
 - Otherwise, the I/O permission bitmap is used
 - One bit for each port; if a bit is set, access is denied
- Access protection for memory mapped I/O via page tables

Bitmap ends with the TSS segment's end, ports with higher numbers must not be accessed.



- History
- x86 programming model
- Address translation
- Protection
- Tasks
- Summary

- x86-64 architecture is highly complex
 - Virtual memory with four or five level paging
 - Page based memory protection
 - I/O-port access protection per task
 - Can execute 16-bit code in IA-32 Protected Mode or 32-bit code in Long Mode
- Rarely used features were removed
 - Segmentation (mostly)
 - Hardware support for task switching
- Still backwards compatible
 - Processor and cores still start up in Real Mode

- Virtualization
 - Virtual 8086 Mode
 - 16-bit application
 - Intel-VT, AMD-V
 - HW support for VMware, VirtualBox, etc.
 - Hypervisor runs in ring -1
- AVX: Advanced Vector Extensions
- TSX: Transactional Synchronization Extensions
- Hybrid Cores